ADOPT-01: Observability Maturity Model

> **Series:** ADOPT | **Notebook:** 1 of 5 | **Created:** March 2026 | **Last Updated:** 03/12/2026

Overview

Observability maturity is not a binary state — organizations progress through distinct levels as their practices, tooling, and culture evolve. This notebook introduces a five-level maturity model for Dynatrace-powered observability, provides assessment criteria for each level, and demonstrates how to use DQL queries to gauge your current position. Understanding where you stand today is the first step toward building a roadmap for improvement.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`storage:logs:read` , `storage:metrics:read` , `storage:entities:read` , `storage:events:read`
Data	At least 24 hours of ingested monitoring data
Audience	Platform engineers, SREs, engineering managers, and leadership stakeholders

1. The Five Maturity Levels

The observability maturity model defines five progressive levels. Each level builds on the previous one, adding capabilities and shifting organizational behavior.

Level	Name	Characteristics	Key Indicator
1	Reactive	Alert-driven, manual troubleshooting, siloed tools	"We find out about problems from users"
2	Proactive	Threshold-based alerts, basic dashboards, some automation	"We detect problems before users report them"
3	Data-Driven	Centralized observability, SLOs defined, DQL-powered analysis	"We make decisions based on telemetry data"
4	Predictive	AI-driven anomaly detection, forecasting, capacity planning	"We predict problems before they occur"
5	Autonomous	Self-healing, automated remediation, closed-loop operations	"The platform resolves problems without human intervention"

Most organizations operate between Level 1 and Level 3. The goal is not necessarily to reach Level 5 everywhere, but to deliberately choose the appropriate level for each service based on business criticality.

2. Level 1 — Reactive Monitoring

Characteristics

- Monitoring is an afterthought, added post-deployment
- Alerts are noisy and frequently ignored
- Troubleshooting relies on SSH-ing into hosts and reading log files
- No standardized observability tooling across teams
- Mean Time to Detect (MTTD) is measured in hours, often reported by end users

Assessment Criteria

Criterion	Level 1 Indicator
Agent coverage	< 50% of hosts instrumented
Data sources	Only infrastructure metrics
Alerting	Static thresholds or none
Dashboards	Ad-hoc, per-team, no standards
Incident process	Manual, tribal knowledge

Dynatrace Features at This Level

- OneAgent deployment on critical hosts
- Basic infrastructure monitoring
- Default Davis AI alerting (often not yet configured)

3. Level 2 — Proactive Monitoring

Characteristics

- Monitoring is part of the deployment checklist
- Threshold-based alerts detect common failure modes
- Centralized dashboards exist but may not be maintained

- Teams respond to problems before user impact escalates
- MTTD is measured in minutes

Assessment Criteria

Criterion	Level 2 Indicator
Agent coverage	50-80% of hosts instrumented
Data sources	Infrastructure + application metrics
Alerting	Threshold-based, some Davis AI
Dashboards	Standardized templates for key services
Incident process	Defined runbooks for top 10 issues

Dynatrace Features at This Level

- Full-stack OneAgent deployment
- Davis AI problem detection enabled
- Custom metric alerting profiles
- Basic synthetic monitoring for key URLs

4. Level 3 — Data-Driven Observability

Characteristics

- Observability data drives architectural and operational decisions
- SLOs are defined and tracked for critical services
- Teams use DQL and Grail for ad-hoc investigation
- Log, trace, and metric data are correlated
- Observability is treated as a platform service

Assessment Criteria

Criterion	Level 3 Indicator
Agent coverage	> 80% of hosts and services instrumented
Data sources	Infrastructure + APM + logs + traces + RUM
Alerting	SLO-based alerting, reduced noise
Dashboards	Business-context dashboards, executive views
Incident process	Data-driven RCA, blameless postmortems

Dynatrace Features at This Level

- Grail data lakehouse with DQL queries
- OpenPipeline for log routing and enrichment
- SLO definitions and burn-rate alerting
- Distributed tracing with span analytics
- Real User Monitoring (RUM) and Session Replay

5. Level 4 — Predictive Observability

Characteristics

- AI-driven anomaly detection replaces static thresholds
- Capacity planning uses forecasting models
- Problems are identified before they cause user impact
- Teams focus on trends and patterns rather than individual alerts

Assessment Criteria

Criterion	Level 4 Indicator
Agent coverage	> 95% with auto-discovery
Data sources	All telemetry types + business events
Alerting	AI-driven, predictive, low false-positive rate
Dashboards	Predictive views, forecasting charts
Incident process	Automated triage and enrichment

Dynatrace Features at This Level

- Davis AI with custom anomaly detection
- Metric forecasting and trend analysis
- Business event correlation
- Ownership and team assignment for entities

6. Level 5 — Autonomous Operations

Characteristics

- Self-healing systems automatically remediate known issues
- Workflows trigger automated responses to problems
- Human intervention is the exception, not the norm
- Continuous optimization is automated

Assessment Criteria

Criterion	Level 5 Indicator
Agent coverage	100% with automated rollout
Data sources	Full telemetry + external integrations
Alerting	Event-driven automation, minimal manual alerts
Dashboards	Real-time executive cockpits
Incident process	Closed-loop: detect → diagnose → remediate → verify

Dynatrace Features at This Level

- Dynatrace Workflows for automated remediation
- AutomationEngine with event-driven triggers
- Configuration-as-code (Monaco) for drift detection
- Full API-driven operations

7. Assessing Your Current Level

The following DQL queries help you assess where your organization stands today. Each query targets a key indicator of maturity.

7.1 Monitored Entity Coverage

Understanding how many entities Dynatrace is monitoring gives a baseline for agent deployment coverage.

```
// Count monitored entities by type to assess coverage breadth
fetch dt.entity.host
| summarize host_count = count()
| append [fetch dt.entity.service | summarize service_count = count()]
| append [fetch dt.entity.process_group | summarize process_group_count =
count()]
| append [fetch dt.entity.application | summarize application_count =
count()]
```

7.2 Data Ingestion Diversity

Mature organizations ingest multiple telemetry types. This query checks which data sources are actively flowing into Grail.

```
// Check log ingestion volume over the last 24 hours
fetch logs, from:-24h
| summarize log_count = count(), distinct_sources = countDistinct(log.source)
```

```
// Check span ingestion to verify distributed tracing is active
fetch spans, from:-24h
| summarize span_count = count(), distinct_services =
countDistinct(dt.entity.service)
```

7.3 Davis AI Problem Detection Activity

Active Davis AI problem detection is a key indicator of Level 2+ maturity. If few or no problems are being detected, it may indicate insufficient instrumentation or misconfigured alerting.

```
// Count Davis problems in the last 7 days by status
fetch dt.davis.problems, from:-7d
| summarize problem_count = count(), by:{event.status}
| sort problem_count desc
```

7.4 Alerting Quality Check

A high ratio of duplicate or frequent events suggests noisy alerting — a characteristic of lower maturity levels.

```
// Assess alerting noise: frequent and duplicate problems vs unique problems
fetch dt.davis.problems, from:-7d
| summarize
    total = count(),
    frequent = countIf(dt.davis.is_frequent_event == true),
    duplicate = countIf(dt.davis.is_duplicate == true)
| fieldsAdd unique = total - frequent - duplicate
| fieldsAdd noise_ratio = round((toDouble(frequent + duplicate) /
toDouble(total)) * 100, decimals: 1)
```

> **Interpreting the noise ratio:**
> - **< 20%** — Healthy alerting (Level 3+)
> - **20-50%** — Moderate noise, review alerting profiles (Level 2)
> - **> 50%** — Significant noise, alerting needs overhaul (Level 1)

8. Mapping Dynatrace Features to Maturity Levels

Use this table to identify which Dynatrace capabilities to adopt next based on your target maturity level.

Feature	L1	L2	L3	L4	L5
OneAgent (Infrastructure)	X	X	X	X	X
OneAgent (Full-Stack APM)		X	X	X	X
Davis AI Problem Detection		X	X	X	X
Synthetic Monitoring		X	X	X	X
Log Management (Grail)			X	X	X
Distributed Tracing (Spans)			X	X	X
OpenPipeline			X	X	X
SLO Definitions			X	X	X
DQL / Notebooks			X	X	X
Real User Monitoring			X	X	X
Business Events				X	X
Davis AI Forecasting				X	X
Ownership & Teams				X	X
Workflows (Automation)					X
Configuration-as-Code (Monaco)					X
AutomationEngine					X

> **Tip:** You do not need to adopt every feature at every level. Focus on the features that address your most pressing gaps.

9. Summary and Next Steps

Key Takeaways

- Observability maturity is a spectrum from reactive (Level 1) to autonomous (Level 5)
- Assessment should be data-driven — use DQL queries to measure coverage, ingestion diversity, and alerting quality
- Not every service needs Level 5 maturity — align investment with business criticality
- Dynatrace features map directly to maturity levels, providing a clear adoption path

Next Steps

- Proceed to **ADOPT-02: Platform Health Assessment** to build a detailed scorecard of your current Dynatrace deployment
- Use the maturity assessment results to prioritize features for adoption
- Share the maturity model with leadership to align on observability investment priorities

---